SkillGauge AI: An Intelligent Interview Assessment System with AI-Driven and Fallback Question Generation

A PROJECT REPORT PHASE-1

Submitted by

Name 1 (Reg 1)

Name 2 (Reg 2)

Name 3 (Reg 3)

In Partial Fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



SRM VALLIAMMAI ENGINEERING COLLEGE

(AN AUTONOMOUS INSTITUTION)

ANNA UNIVERSITY: CHENNAI 600 025

ANNA UNIVERSITY: CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified this project report **"SkillGauge AI: An Intelligent Interview Assessment System with AI-Driven and Fallback Question Generation"** is the bonafide work of **Name (Reg)** who carried out the work under my Supervision.

SIGNATURE

Dr. G. KUMARESAN B.E., M.E., Ph.D.,

ASSISTANT PROFESSOR (Sr. G.),

Department of CSE,

SRM Valliammai Engineering College,

Kattankulathur – 603 203.

SIGNATURE

Dr. B. VANATHI, M.E., Ph.D.,

PROFESSOR & HEAD,

Department of CSE,

SRM Valliammai Engineering College,

Kattankulathur – 603 203.

Submitted for the university examination held on_____ at SRM Valliammai Engineering College, Kattankulathur.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

We sincerely express our gratitude in depth to our esteemed Founder Chairman & Chancellor **Dr. T. R. Paarivendhar, Thiru. Ravi Pachamoothu** Chairman, **Mrs. Padmapriya Ravi** Vice Chairman, **Ms. R. Harini** Correspondent, SRM Valliammai Engineering College for the patronage on our welfare rooted in the academic year.

We express our sincere gratitude to our respected Director **Dr. B. Chidhambararajan, M.E., Ph.D.,** for his constant encouragement, which has been our motivation to strive towards excellence.

We thank our sincere Principal **Dr. M. Murugan, M.E., Ph.D.,** for his constant encouragement, which has been our motivation to strive towards excellence.

We thank our Vice Principal **Dr. S. Visalakshi, M.E., Ph.D.,** for his constant encouragement, which has been our motivation to strive towards excellence.

We extend our hand of thanks to our Head of the Department**, Dr. B. Vanathi, M.E., Ph.D.,** Professor for her unstinted support.

We are grateful to our Project Coordinator and internal guide **Professor Name** Assistant Professor without whose invaluable guidance and encouragement, this project would not have been a success.

We also like to thank all Teaching and non-teaching staff members of our department, for their support during the project. We finally thank our friends and family for their support during the project.

ABSTRACT

Technical interview assessment is a critical process in both recruitment and academic environments for evaluating the knowledge, problem-solving ability, and job readiness of candidates and students. Conventional interview assessment methods rely on manual question preparation and human evaluation, which are often time-consuming, inconsistent, and subject to bias. Moreover, students and job seekers lack reliable platforms to independently assess their technical capabilities and interview preparedness in a structured manner.

SkillGauge AI is proposed as an intelligent interview assessment system that supports both job interview screening and self-evaluation of technical skills for students. The system enables users to either manually enter their technical skills or upload a resume in PDF format, from which relevant skills are extracted using Natural Language Processing techniques. Based on the identified skill set, the system generates domain-specific multiple-choice interview questions using a Generative AI model integrated through the Gemini API. To ensure uninterrupted assessment, a curated fallback question bank is incorporated to handle scenarios where AI services are unavailable.

The assessment is conducted through a web-based interface developed using Streamlit, where users answer the generated questions in real time. The system automatically evaluates responses, calculates scores, and performs performance analysis using data analytics techniques. Detailed assessment reports are generated in PDF format and delivered via email, enabling users to review their strengths and areas for improvement. SkillGauge AI improves efficiency, scalability, and fairness in interview assessments, making it suitable for recruitment processes, training institutions, and students seeking to evaluate and enhance their technical capabilities.

ABSTRACT(TAMIL)

தொழில்நுட்ப நேர்முகத் தேர்வு மதிப்பீடு என்பது வேலைவாய்ப்பு மற்றும் கல்வி சூழல்களில் மாணவர்கள் மற்றும் வேட்பாளர்களின் அறிவு, சிக்கல் தீர்க்கும் திறன் மற்றும் வேலைத் தயார்திறனை மதிப்பிடுவதற்கான முக்கியமான செயல்முறையாகும். பாரம்பரிய நேர்முகத் தேர்வு முறைகள் பெரும்பாலும் கைமுறையாக கேள்விகள் தயாரித்தல் மற்றும் மனித மதிப்பீடுகளை சார்ந்துள்ளதால் அதிக நேரம் எடுத்துக் கொள்வதுடன், ஒரே மாதிரியான மதிப்பீடு இல்லாமலும், பாகுபாடுகளும் ஏற்படுகின்றன. மேலும், மாணவர்கள் மற்றும் வேலை தேடுபவர்கள் தங்களின் தொழில்நுட்ப திறனை தனிப்பட்ட முறையில் கட்டமைக்கப்பட்ட முறையில் மதிப்பிடுவதற்கான நம்பகமான தளங்கள் குறைவாக உள்ளன.

SkillGauge AI என்பது வேலைவாய்ப்பு நேர்முகத் தேர்வுகளுக்கும் மாணவர்களின் தன்னிலைத் திறன் மதிப்பீட்டிற்கும் ஆதரவளிக்கும் அறிவார்ந்த நேர்முகத் தேர்வு மதிப்பீட்டு அமைப்பாக முன்மொழியப்படுகிறது. பயனர்கள் தங்களின் தொழில்நுட்ப திறன்களை கைமுறையாக உள்ளிடவோ அல்லது PDF வடிவிலான சுயவிவரத்தை பதிவேற்றவோ முடியும். பதிவேற்றப்பட்ட சுயவிவரத்திலிருந்து தொடர்புடைய திறன்கள் இயற்கை மொழி செயலாக்க (NLP) நுட்பங்களின் மூலம் பிரித்தெடுக்கப்படுகின்றன. கண்டறியப்பட்ட திறன்களின் அடிப்படையில், Generative Artificial Intelligence பயன்படுத்தி தொழில்நுட்ப அடிப்படையிலான பல்தேர்வு நேர்முகக் கேள்விகள் உருவாக்கப்படுகின்றன. செயற்கை நுண்ணறிவு சேவைகள் கிடைக்காத சூழ்நிலைகளிலும் மதிப்பீடு தொடரும் வகையில் மாற்று கேள்வி களஞ்சியம் (Fallback Question Bank) இணைக்கப்பட்டுள்ளது.

இந்த மதிப்பீடு Streamlit மூலம் உருவாக்கப்பட்ட இணைய அடிப்படையிலான பயன்பாட்டின் வழியாக நடத்தப்படுகிறது. பயனர்களின் பதில்கள் தானியங்கியாக மதிப்பிடப்பட்டு, மதிப்பெண்கள் கணக்கிடப்படுகின்றன மற்றும் செயல்திறன் பகுப்பாய்வு மேற்கொள்ளப்படுகிறது. விரிவான மதிப்பீட்டு அறிக்கைகள் PDF வடிவில் உருவாக்கப்பட்டு மின்னஞ்சல் மூலம் அனுப்பப்படுகின்றன. SkillGauge AI நேர்முகத் தேர்வு மதிப்பீட்டில் திறன், அளவீட்டு வசதி மற்றும் நியாயத்தன்மையை மேம்படுத்தி, கல்வி நிறுவனங்கள், பயிற்சி மையங்கள் மற்றும் வேலைவாய்ப்பு செயல்முறைகளுக்கு ஏற்ற நம்பகமான தீர்வாக விளங்குகிறது.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
	ABSTRACT(ENGLISH)	v
	ABSTRACT(TAMIL)	vi
	LIST OF FIGURES	xi
1	**INTRODUCTION**	1
	1.1 SYSTEM OVERVIEW	1
	1.2 PURPOSE OF THE PROJECT	1
	1.3 OBJECTIVE OF THE PROJECT	2
	1.4 EXISTING SYSTEM	3
	1.5 PROPOSED SYSTEM	3
2	**LITERATURE SURVEY**	5
	2.1 INTRODUCTION	5
	2.2 AI-BASED INTERVIEW AND SKILL ASSESSMENT SYSTEMS	5
	2.3 AI-DRIVEN DECISION SUPPORT SYSTEMS FOR ASSESSMENT	6
	2.4 AUTOMATED FEEDBACK AND REPORT GENERATION SYSTEMS	7
	2.5 WEB-BASED INTERVIEW AND ASSESSMENT APPLICATIONS	7
	2.6 SUMMARY OF THE LITERATURE SURVEY	8
3	**SYSTEM REQUIREMENTS AND SPECIFICATION**	9

3.1 SOFTWARE REQUIREMENTS 9
3.1.1 Programming Language – Python 9
3.1.2 Frontend Framework – Streamlit 9
3.1.3 Document Processing Libraries 10
3.1.4 Temporary File Handling And Storage 10
3.1.5 Operating System Compatibility 11
3.1.6 Development Environment And Tools 11
3.1.7 Version Control System 11
3.1.8 Security And Reliability 12
3.2 HARDWARE REQUIREMENTS 12
3.2.1 Processor 12
3.2.2 Main Memory (RAM) 13
3.2.3 Storage Requirements 13
3.2.4 Input Devices 14
3.2.5 Output Devices 14
3.2.6 Network Requirements 14
3.2.7 End-User System Configuration 15
3.2.8 Scalability and Performance Considerations 15
3.3 TECHNIQUES USED 15
3.3.1 Generative AI–Based Evaluation Technique 15
3.3.2 Natural Language Processing Technique 16
3.3.3 Prompt Engineering Technique 16
3.3.4 Web-Based Application Development 17
Technique
3.3.5 Automated Report Generation Technique 17
3.3.6 Email Automation Technique 18
3.3.7 Input Validation and Error Handling 18
Technique
3.3.8 Session State Management Technique 18

3.3.9 Performance Optimization Technique 19

4 **SYSTEM DESIGN** 20

4.1 INTRODUCTION 20

4.2 SYSTEM ARCHITECTURE 20

4.3 USE CASE DIAGRAM 22

4.4 CLASS DIAGRAM 23

4.5 COLLABARATIVE DIAGRAM 24

4.6 SEQUENCE DIAGRAM 25

5 **SYSTEM IMPLEMENTATION** 28

5.1 INTRODUCTION 28

5.2 IMPLEMENTATION ENVIRONMENT 28

5.3 IMPLEMENTATION METHODOLOGY 29

5.4 MODULE-WISE IMPLEMENTATION 30

5.4.1 User Interface Module 30

5.4.2 Input Validation Module 31

5.4.3 AI Evaluation Module 31

5.4.4 Report Generation Module 31

5.4.5 Decision Support Module 32

5.4.6 Output File Generation Module 32

5.4.7 Temporary File Management Module 32

5.5 ERROR HANDLING AND EXCEPTION MANAGEMENT 33

5.6 SYSTEM EXECUTION FLOW 33

6 **SYSTEM TESTING** 34

6.1 INTRODUCTION 34

6.2 OBJECTIVES OF SYSTEM TESTING 34
6.3 TESTING STRATEGY 35

6.4 TYPES OF TESTING 35
6.4.1 Unit Testing 35
6.4.2 Integration Testing 35
6.4.3 Functional Testing 36
6.4.4 Input Validation Testing 36
6.4.5 Performance Testing 36
6.4.6 Usability Testing 36
6.5 TEST CASE DESCRIPTION 37
6.6 TEST RESULTS AND ANALYSIS 38

7 **CONCLUSION AND FUTURE WORK** 39
7.1 Conclusion 39
7.2 Future Work 40
APPENDIX I – SAMPLE CODE 41
APPENDIX II – SAMPLE OUTPUT 67
REFERENCES 72

TABLE OF FIGURES

Figure No.	List of Figures	Page No.
4.2	System Architecture	20
4.3	Use Case Diagram	22
4.4	Class Diagram	23
4.5	Collaborative Diagram	24
4.6	Sequence Diagram	25

CHAPTER 1

INTRODUCTION

1.1 SYSTEM OVERVIEW

Technical interview assessment has become an essential component in both educational and recruitment environments for evaluating the technical competence and problem-solving abilities of individuals. With the rapid growth of technology-oriented roles, there is an increasing need for structured, scalable, and unbiased methods to assess candidates efficiently. Traditional interview practices often struggle to maintain consistency and fairness, especially when dealing with large numbers of candidates with diverse skill sets.

SkillGauge AI is an intelligent interview assessment system designed to automate the process of evaluating technical skills. The system allows users to either manually enter their skills or upload a resume in PDF format. Using Natural Language Processing techniques, relevant technical skills are extracted and analyzed. Based on these skills, the system generates domain-specific multiple-choice interview questions using Generative Artificial Intelligence.

To ensure reliability and uninterrupted operation, SkillGauge AI integrates a fallback question bank that is activated when AI services are unavailable. The system conducts automated evaluation, calculates scores, performs performance analysis, and generates detailed assessment reports. These reports are delivered via email, while assessment records are maintained for administrative review, making the system efficient, scalable, and unbiased.

1.2 PURPOSE OF THE PROJECT

The purpose of this project is to address the limitations of conventional interview assessment methods that rely heavily on manual processes and subjective decision-making. Manual interview preparation and evaluation demand significant

time and effort from recruiters and faculty members, often resulting in inconsistent assessment standards. These challenges reduce the effectiveness of interviews in accurately measuring a candidate's true technical capability.

SkillGauge AI aims to provide a structured and automated interview assessment platform that can be used both for recruitment screening and student self-evaluation. By generating skill-based questions dynamically, the system ensures that each assessment is tailored to the individual's technical background. This helps students identify their strengths and weaknesses while preparing for real-world interviews.

Another key purpose of the project is to enhance transparency and documentation in interview assessments. Automated report generation and email communication ensure that results are clearly communicated and properly recorded. By reducing manual workload and improving consistency, the system supports better decision-making in educational institutions and recruitment processes.

1.3 OBJECTIVE OF THE PROJECT

One of the primary objectives of the SkillGauge AI project is to design and develop an intelligent system capable of automating technical interview assessments. The system seeks to eliminate the dependency on manual question preparation and subjective evaluation by introducing AI-driven question generation and automated scoring mechanisms.

Another objective is to ensure fairness, reliability, and scalability in the assessment process. By extracting skills from resumes and generating domain-specific questions, the system provides personalized assessments for each user. The inclusion of a fallback question bank ensures uninterrupted functionality, even when AI services are unavailable.

Additionally, the project aims to provide comprehensive performance analysis and reporting. Automated score calculation, visual analytics, PDF report generation,

and email delivery help users and administrators easily interpret results. The admin dashboard further supports monitoring and record maintenance, making the system suitable for large-scale deployment.

1.4 EXISTING SYSTEM

In the existing system, interview assessments are primarily conducted through manual processes involving human interviewers. Questions are prepared based on individual experience or fixed templates, and evaluation is performed subjectively during or after interviews. This approach often lacks standardization and consistency across different candidates and evaluators.

Manual interview systems are time-consuming and difficult to scale, especially when evaluating a large number of candidates. The same set of questions is frequently used for all candidates, regardless of their technical background, which reduces the effectiveness of skill evaluation. Additionally, maintaining records and generating detailed performance reports require extra effort and are prone to human error.

Existing systems also suffer from bias and lack of transparency. Since evaluation depends on human judgment, assessment outcomes may vary significantly. The absence of automated analytics, structured feedback, and centralized data storage limits the ability to track performance trends and improve the overall interview process.

1.5 PROPOSED SYSTEM

The proposed system, SkillGauge AI, introduces an automated and intelligent approach to technical interview assessment. The system dynamically generates interview questions based on a candidate's technical skills using Generative Artificial Intelligence. This ensures that assessments are personalized, relevant, and aligned with individual skill sets.

To enhance reliability, the proposed system includes a fallback question bank that allows assessments to continue even when AI services are unavailable. Automated evaluation and scoring mechanisms eliminate subjectivity and ensure consistent assessment standards. Performance analytics and visualizations provide clear insights into candidate capabilities.

Furthermore, the system generates detailed PDF assessment reports and delivers them via email, improving transparency and documentation. An integrated admin dashboard enables efficient monitoring of assessment records and performance trends. Overall, the proposed system reduces manual effort, improves scalability, and provides a fair and efficient interview assessment solution for both educational and recruitment environments.

CHAPTER 2

LITERATURE SURVEY

2.1 INTRODUCTION

The rapid advancement of Artificial Intelligence has significantly influenced the development of automated assessment and evaluation systems across education and recruitment domains. Traditional interview and skill evaluation methods are increasingly being replaced or supported by intelligent systems that aim to improve efficiency, consistency, and scalability. Researchers have explored various AI-driven techniques to automate question generation, candidate evaluation, and performance analysis.

Recent studies highlight the role of Generative Artificial Intelligence and Natural Language Processing in transforming assessment systems. These technologies enable systems to understand textual input, generate context-aware questions, and evaluate responses in a structured manner. The growing adoption of AI in assessment applications demonstrates its potential to reduce human workload and subjectivity.

This chapter presents a review of existing literature related to AI-based assessment systems, decision support mechanisms, automated feedback generation, and web-based evaluation platforms. The survey helps identify research gaps and provides a foundation for the design of the proposed SkillGauge AI system.

2.2 AI-BASED INTERVIEW AND SKILL ASSESSMENT SYSTEMS

Several researchers have proposed AI-based systems to automate interview and skill assessment processes. These systems utilize machine learning, deep learning, and natural language processing techniques to evaluate candidate performance objectively. Studies have shown that AI-driven assessments can

effectively measure technical knowledge and problem-solving ability while maintaining consistency across evaluations.

Generative AI models, particularly Large Language Models, have gained attention for their ability to generate interview questions dynamically based on candidate profiles. Research indicates that AI-generated questions can adapt to different skill levels and domains, offering personalized assessments. Such approaches help overcome the limitations of static question banks commonly used in traditional interviews.

Despite their advantages, existing AI-based interview systems often depend entirely on AI services, making them vulnerable to service unavailability or high computational costs. This limitation highlights the importance of incorporating reliability mechanisms, such as fallback question banks, which are addressed in the proposed SkillGauge AI system.

2.3 AI-DRIVEN DECISION SUPPORT SYSTEMS FOR ASSESSMENT

Decision support systems (DSS) play a crucial role in assisting evaluators and recruiters in making informed decisions based on assessment data. AI-driven DSS integrate data analytics and intelligent models to analyze candidate performance and generate recommendations. In interview assessment scenarios, such systems help reduce bias and improve decision accuracy.

In educational and recruitment contexts, AI-based decision support systems have been used to rank candidates, classify performance levels, and suggest improvement strategies. These systems leverage structured evaluation metrics and automated scoring mechanisms to support consistent decision-making. Research shows that AI-assisted decisions outperform purely manual approaches in terms of efficiency and objectivity.

However, many existing decision support systems lack transparency and detailed reporting mechanisms. The absence of explainable evaluation results can reduce user trust. SkillGauge AI addresses this gap by generating detailed performance reports and analytics that clearly justify evaluation outcomes.

2.4 AUTOMATED FEEDBACK AND REPORT GENERATION SYSTEMS

Automated feedback systems have been widely studied for their ability to provide timely and structured responses to users. In assessment applications, such systems generate feedback based on performance metrics, enabling users to identify strengths and weaknesses. Research emphasizes that immediate feedback enhances learning outcomes and interview preparedness.

Several studies focus on automated report generation using programmatic document creation techniques. These systems convert evaluation results into structured reports, improving documentation and record maintenance. Automated reports also support transparency by clearly presenting assessment criteria and outcomes.

Despite these advancements, many feedback systems are limited to basic score reporting and lack comprehensive performance analysis. SkillGauge AI enhances automated feedback by integrating performance analytics, visual representations, and detailed PDF reports delivered through email communication.

2.5 WEB-BASED INTERVIEW AND ASSESSMENT APPLICATIONS

Web-based assessment platforms have gained popularity due to their accessibility and scalability. Such platforms allow users to participate in assessments remotely through browser-based interfaces. Research highlights that web-based systems reduce infrastructure requirements and enable large-scale assessments.

Modern web-based interview systems integrate AI models, data analytics, and visualization tools to provide interactive assessment experiences. Frameworks like Streamlit have simplified the development of rapid, user-friendly interfaces for AI-powered applications. These platforms support real-time interaction, progress tracking, and instant result display.

However, many existing web-based systems focus primarily on usability and lack robust evaluation logic or reliability features. The proposed SkillGauge AI system combines an intuitive web interface with intelligent backend processing, ensuring both usability and dependable assessment functionality.

2.6 SUMMARY OF THE LITERATURE SURVEY

The literature survey reveals that AI-based interview and skill assessment systems have made significant progress in automating evaluation processes. Generative AI, NLP, and decision support systems have demonstrated strong potential in improving consistency, scalability, and objectivity in assessments.

However, existing systems often suffer from limitations such as complete dependency on AI services, lack of fallback mechanisms, insufficient feedback detail, and limited administrative analytics. These gaps highlight the need for a more reliable and comprehensive assessment framework.

Based on the reviewed literature, SkillGauge AI is proposed as a system that integrates AI-driven question generation, automated evaluation, fallback reliability mechanisms, detailed reporting, and web-based accessibility. The proposed approach addresses the identified research gaps and provides a robust solution for modern interview assessment needs.

CHAPTER 3

SYSTEM REQUIREMENTS AND SPECIFICATION

3.1 SOFTWARE REQUIREMENTS

3.1.1 Programming Language – Python

Python is used as the primary programming language for the development of the SkillGauge AI system. It offers simplicity, readability, and extensive library support, making it suitable for implementing AI-based applications and web systems. Python enables rapid development and easy integration of various modules used in the project.

The project utilizes Python for backend logic, AI integration, data processing, evaluation mechanisms, and report generation. Libraries such as Pandas, Matplotlib, PyPDF2, ReportLab, and OpenPyXL are leveraged to handle data analytics, document processing, and file management efficiently.

Python's compatibility with AI frameworks and APIs, including the Gemini Generative AI API, allows seamless integration of intelligent question generation and skill extraction features. Its cross-platform nature ensures that the application can run consistently across different operating systems.

3.1.2 Frontend Framework – Streamlit

Streamlit is used as the frontend framework to develop the web-based user interface of SkillGauge AI. It enables the creation of interactive and responsive interfaces with minimal code, making it ideal for rapid prototyping and deployment of AI-driven applications.

The framework supports user inputs such as text fields, file uploads, buttons, and visual components like charts and metrics. These features are utilized to build

candidate assessment forms, display interview questions, show progress indicators, and present performance analytics.

Streamlit also provides built-in session state management, which is used to track user progress during the interview assessment. Its seamless integration with Python backend logic ensures real-time interaction between frontend components and backend processing.

3.1.3 Document Processing Libraries

Document processing libraries play a vital role in handling resumes and assessment reports within the system. PyPDF2 is used to extract textual content from uploaded PDF resumes for skill identification and analysis.

ReportLab is employed to generate structured PDF assessment reports that include questions, user responses, correct answers, and performance scores. This ensures professional documentation of assessment outcomes.

OpenPyXL is used to manage Excel-based storage of assessment records. These libraries collectively enable efficient document handling, data storage, and automated report generation within the application.

3.1.4 Temporary File Handling and Storage

The system requires temporary storage to handle uploaded resumes and generated reports during execution. Temporary directories are used to store intermediate files securely while ensuring proper cleanup after processing.

File handling operations are managed using Python's built-in file system libraries. This approach minimizes persistent storage usage and reduces the risk of unauthorized file access.

Generated PDF reports and Excel files are stored in structured directories to ensure organized data management. This design supports easy retrieval and administrative analysis while maintaining system reliability.

3.1.5 Operating System Compatibility

SkillGauge AI is designed to be platform-independent and compatible with multiple operating systems. The application can run on Windows, Linux, and macOS without requiring system-specific modifications.

Python and Streamlit provide cross-platform support, ensuring consistent performance across different environments. The use of standard libraries further enhances portability.

This compatibility allows the system to be deployed on personal computers, institutional servers, or cloud-based environments, making it flexible for various usage scenarios.

3.1.6 Development Environment and Tools

The development of SkillGauge AI is carried out using modern Integrated Development Environments (IDEs) such as Visual Studio Code and PyCharm. These tools provide features like syntax highlighting, debugging, and code management.

Additional tools such as Jupyter Notebook may be used during experimentation and testing of AI prompts and data processing logic. These tools support efficient development and testing workflows.

The use of virtual environments ensures proper dependency management and prevents conflicts between project libraries. This contributes to a stable and maintainable development process.

3.1.7 Version Control System

A version control system is essential for managing code changes and maintaining project history. Git is used to track modifications, manage different versions, and collaborate effectively during development.

Version control helps in identifying bugs, rolling back changes, and maintaining a stable codebase. It also supports future enhancements and scalability of the system.

Repositories can be hosted on platforms such as GitHub or GitLab, enabling secure storage and easy sharing of the project source code.

3.1.8 Security and Reliability

Security and reliability are critical considerations in the SkillGauge AI system. User inputs and uploaded documents are handled securely to prevent unauthorized access and data misuse.

The system incorporates a fallback question bank to ensure uninterrupted assessment in case of AI service unavailability. This enhances reliability and ensures consistent system performance.

Email communication and data storage mechanisms are implemented carefully to avoid data loss and errors. These measures ensure that the system remains dependable and trustworthy for users and administrators.

3.2 HARDWARE REQUIREMENTS

3.2.1 Processor

The processor is a critical hardware component required for executing the SkillGauge AI application efficiently. Since the system involves AI-based processing, document handling, and real-time user interaction, a moderately powerful processor is sufficient to support these operations smoothly.

A minimum of an Intel Core i3 or equivalent processor is recommended for basic execution, while Intel Core i5 or higher processors are preferred for improved performance. Multi-core processors enhance the system’s ability to handle concurrent tasks such as AI interaction, evaluation logic, and report generation.

For deployment in institutional or multi-user environments, higher-performance processors ensure faster response times and better scalability. The processor requirement ensures that the system runs efficiently without noticeable delays during assessments.

3.2.2 Main Memory (RAM)

Main memory plays a vital role in managing application execution, data processing, and user interactions. SkillGauge AI requires sufficient RAM to handle resume parsing, AI-generated content, and session management during assessments.

A minimum of 4 GB RAM is required to run the application smoothly for individual users. However, 8 GB RAM or higher is recommended for improved multitasking, especially when handling multiple assessments or administrative analytics.

Adequate RAM ensures stable system performance, prevents application crashes, and allows smooth rendering of charts, reports, and interactive UI components.

3.2.3 Storage Requirements

Storage is required to maintain application files, temporary uploaded documents, generated reports, and assessment records. SkillGauge AI uses local storage for managing PDF reports and Excel-based result files.

A minimum of 20 GB of free disk space is sufficient for basic operation, including storing reports and logs. Solid State Drives (SSD) are preferred over traditional hard drives to improve file access speed and overall system responsiveness.

Efficient storage management ensures quick access to reports and reliable long-term record maintenance for administrative analysis.

3.2.4 Input Devices

Input devices are necessary for users to interact with the SkillGauge AI system. Standard input devices such as a keyboard and mouse are sufficient for entering skills, navigating the interface, and completing assessments.

For resume-based evaluation, users upload PDF documents using file input controls provided by the system. This requires basic file handling support through standard input devices.

These input mechanisms ensure ease of use and accessibility for both students and recruiters using the system.

3.2.5 Output Devices

Output devices are required to display assessment questions, results, performance analytics, and reports. A standard computer monitor is sufficient to view the web-based interface and visual charts.

Generated PDF reports can be viewed on-screen or printed using standard printing devices. This supports offline review and documentation when required.

Clear output display ensures better user experience and effective interpretation of assessment results.

3.2.6 Network Requirements

Network connectivity is essential for accessing AI services, sending email reports, and supporting web-based interaction. A stable internet connection is required to communicate with the Generative AI API and SMTP email services.

A minimum broadband connection with standard upload and download speeds is sufficient for smooth operation. Reliable connectivity ensures quick response times during question generation and report delivery.

In offline or low-connectivity scenarios, the fallback question mechanism allows assessments to continue, enhancing system robustness.

3.2.7 End-User System Configuration

The end-user system must meet basic hardware specifications to run the application efficiently. A personal computer or laptop with a supported operating system, sufficient RAM, and a modern web browser is adequate.

No specialized hardware or external devices are required, making the system easily accessible to students and recruiters. This reduces deployment cost and complexity.

The simple configuration ensures that SkillGauge AI can be widely adopted without additional infrastructure investment.

3.2.8 Scalability and Performance Considerations

Scalability is an important factor when deploying SkillGauge AI in institutional or recruitment environments. The system is designed to handle increasing numbers of users by leveraging efficient software and hardware resources.

Performance can be enhanced by upgrading hardware components such as RAM, processor, and storage. For large-scale deployment, the system can be hosted on servers or cloud platforms.

These considerations ensure that the system remains responsive, reliable, and efficient as user demand grows.

3.3 TECHINQUES USED

3.3.1 Generative AI–Based Evaluation Technique

Generative Artificial Intelligence is used in SkillGauge AI to dynamically generate technical interview questions based on the user's skill profile. The system leverages a large language model to understand the provided skills and produce

relevant, domain-specific multiple-choice questions. This approach enables personalized assessment instead of using static question sets.

The AI-based technique ensures adaptability by generating questions across various technical domains such as programming, databases, web technologies, and cloud computing. This improves the relevance and effectiveness of the interview assessment by aligning questions with individual capabilities.

To ensure reliability, the system incorporates a fallback question bank that activates when AI services are unavailable. This hybrid approach balances intelligence and robustness, ensuring uninterrupted evaluation while maintaining assessment quality.

3.3.2 Natural Language Processing Technique

Natural Language Processing (NLP) techniques are used to extract meaningful technical skills from uploaded resume documents. The system processes PDF resumes and extracts textual content, which is analyzed to identify programming languages, frameworks, tools, and domains.

By applying NLP-based text analysis, the system converts unstructured resume data into structured skill information. This allows automatic generation of relevant interview questions without requiring manual skill entry.

The NLP technique enhances usability and reduces user effort, particularly for job seekers and students who prefer resume-based assessments. It also improves accuracy in skill identification, leading to more precise interview evaluations.

3.3.3 Prompt Engineering Technique

Prompt engineering plays a crucial role in controlling the behavior and output of the Generative AI model. Carefully structured prompts are designed to ensure that the AI generates interview questions in a consistent and valid format.

The prompts enforce constraints such as question count, multiple-choice structure, and inclusion of correct answers. This minimizes malformed responses and improves the reliability of AI-generated content.

By refining prompts, the system achieves predictable and high-quality outputs suitable for automated evaluation. This technique ensures seamless integration between AI-generated content and the system's evaluation logic.

3.3.4 Web-Based Application Development Technique

SkillGauge AI is implemented as a web-based application using the Streamlit framework. This technique enables rapid development of an interactive user interface while maintaining seamless integration with backend processing.

The web-based approach allows users to access the system through standard web browsers without requiring additional software installation. It supports real-time interaction, progress tracking, and dynamic content rendering.

This technique improves accessibility and scalability, making the system suitable for educational institutions, training centers, and recruitment environments.

3.3.5 Automated Report Generation Technique

Automated report generation is used to document interview assessment results in a structured and professional format. The system generates detailed PDF reports that include questions, responses, correct answers, scores, and performance summaries.

Programmatic document generation techniques ensure consistency and eliminate manual reporting effort. The reports serve as official records for both users and administrators.

This technique improves transparency and accountability by providing clear documentation of assessment outcomes.

3.3.6 Email Automation Technique

Email automation is implemented to deliver assessment reports directly to users. The system integrates SMTP-based email services to send generated PDF reports automatically upon assessment completion.

This technique ensures timely communication and eliminates the need for manual report distribution. Users receive instant feedback on their performance through email notifications.

Automated email delivery enhances user experience and supports efficient communication in large-scale assessment scenarios.

3.3.7 Input Validation and Error Handling Technique

Input validation techniques are applied to ensure that users provide valid and complete information before starting an assessment. This includes validating skill input, email addresses, and uploaded resume files.

Error handling mechanisms are implemented to manage unexpected issues such as invalid file formats, missing inputs, or AI service failures. Informative messages guide users in correcting errors.

These techniques improve system stability, prevent crashes, and ensure smooth user interaction.

3.3.8 Session State Management Technique

Session state management is used to maintain user progress throughout the interview assessment. The system tracks answered questions, selected options, and current assessment state.

This technique ensures continuity during multi-step interactions such as question navigation and submission. It prevents data loss during page refreshes or navigation events.

Session management enhances reliability and provides a seamless assessment experience for users.

3.3.9 Performance Optimization Technique

Performance optimization techniques are applied to ensure fast response times and efficient resource utilization. Lightweight libraries and efficient data handling mechanisms are used to minimize processing overhead.

Caching and structured data storage improve retrieval speed for assessment records and reports. Optimized code execution ensures smooth performance even with multiple users.

These techniques enable the system to scale effectively while maintaining responsiveness and reliability.

CHAPTER 4

SYSTEM DESIGN

4.1 INTRODUCTION

System design defines the structural and functional blueprint of the SkillGauge AI interview assessment platform. It describes how various components interact to deliver automated interview question generation, assessment, evaluation, and reporting. The design focuses on reliability, scalability, and user-friendliness while ensuring unbiased and consistent assessment.

A key aspect of the system design is the hybrid evaluation approach that combines AI-driven question generation with a curated fallback question bank. This ensures that the system remains functional even when Generative AI services are unavailable. The design prioritizes uninterrupted assessment as a critical requirement.

This chapter explains the system architecture, use case interactions, class relationships, collaboration between modules, and the sequence of operations involved in the assessment process. The design ensures seamless coordination between AI modules, fallback mechanisms, evaluation logic, and reporting components.

4.2 SYSTEM ARCHITECTURE

The system architecture of SkillGauge AI is designed as a modular and intelligent interview assessment framework that integrates AI-driven processing with a reliable fallback mechanism. As shown in the architecture diagram, the system begins with the Candidate Portal, where users can either enter their technical skills manually or upload their resume in PDF format. These inputs form the primary data source for the assessment process. The architecture ensures

flexibility by supporting both manual skill input and automated resume-based skill extraction.

Once the skills are collected, the data is passed to the Generative AI module, which is responsible for automated question generation. The system continuously checks AI service availability before generating questions. When the AI service is active, domain-specific multiple-choice interview questions are generated dynamically based on the extracted skills. A key highlight of this architecture is the Predefined Question Bank, which acts as a fallback mechanism. If the AI service becomes unavailable or fails to generate valid questions, the system automatically switches to this curated question bank, ensuring uninterrupted interview assessments.

The generated questions are then forwarded to the Automated Evaluation module, where candidate responses are compared with correct answers and scores are calculated automatically. The results are processed by the Result Generation and Analysis module, which produces detailed PDF reports and visual performance analytics. These reports are delivered to candidates via email, while assessment records are stored for administrative review. An Administrative Dashboard enables recruiters or administrators to analyze performance trends and assessment data. Overall, the architecture ensures reliability, scalability, and continuity by combining AI intelligence with a robust fallback question mechanism.

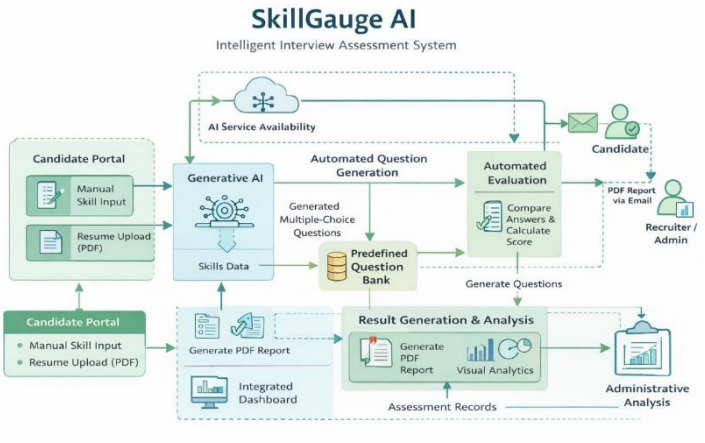


Figure 4.2 System Architecture

4.3 USE CASE DIAGRAM

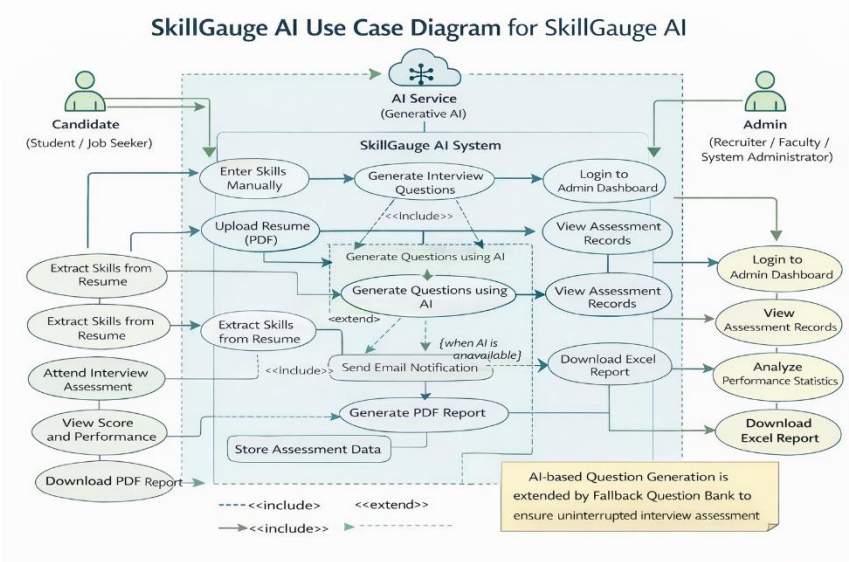


Figure 4.3 Use Case Diagram

The use case diagram of SkillGauge AI – Intelligent Interview Assessment System illustrates the interaction between different actors and the system functionalities involved in conducting automated interview assessments. The diagram primarily includes three actors: Candidate (Student/Job Seeker), Admin (Recruiter/Faculty/System Administrator), and the AI Service (Generative AI). These actors interact with the SkillGauge AI system to perform various assessment-related operations.

The Candidate is the primary user of the system. The candidate can either enter technical skills manually or upload a resume in PDF format. When a resume is uploaded, the system performs skill extraction using natural language processing techniques. After skill identification, the candidate initiates the interview assessment, during which the system generates multiple-choice questions. The use case Generate Interview Questions includes the process of checking AI service availability. If the AI service is available, questions are generated using Generative AI; otherwise, the process is extended to fetch questions from the fallback question bank, ensuring uninterrupted assessments. The candidate then submits answers, views scores and performance analysis, downloads the PDF report, and receives the report via email.

The AI Service (Generative AI) acts as a supporting actor responsible for dynamically generating interview questions based on the candidate's skill set. The use case Generate Questions using AI depends on the AI service availability. The

diagram clearly highlights the extend relationship, where AI-based question generation is extended by the fallback question bank when the AI service is unavailable. This design ensures system reliability and continuous operation without dependency failures.

The Admin interacts with the system through the Admin Dashboard. The admin can log in, view assessment records, analyze performance statistics, and download consolidated Excel reports. The system also includes use cases for storing assessment data, generating PDF reports, and sending email notifications, which are included as part of the evaluation and reporting workflow. Overall, the use case diagram demonstrates how SkillGauge AI integrates automated evaluation, AI intelligence, fallback mechanisms, reporting, and administrative monitoring into a single, scalable interview assessment platform.

4.4 CLASS DIAGRAM

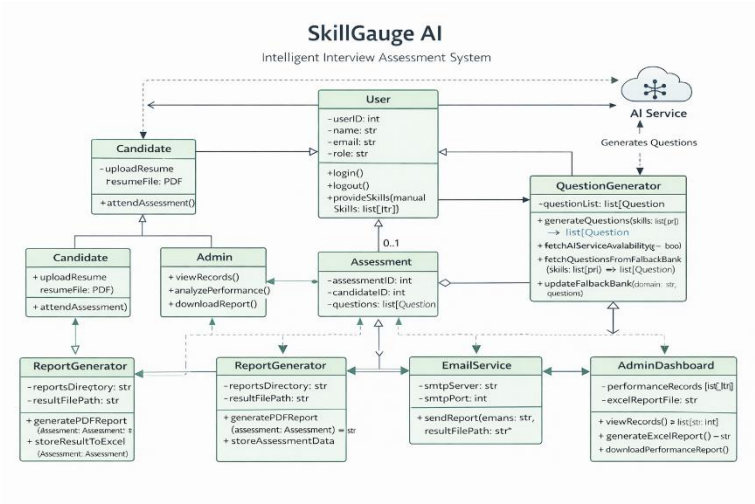


Figure 4.4 Class Diagram

The class diagram of SkillGauge AI – Intelligent Interview Assessment System represents the static structure of the system by illustrating the classes, their attributes, methods, and relationships. The core class in the system is the User class, which acts as a parent entity for different types of users. It contains basic attributes such as user ID, name, email, and role, along with common operations like login, logout, and skill submission. This abstraction ensures reusability and consistency across different user roles.

The Candidate class extends the User class and represents students or job seekers using the system for interview assessment. This class includes operations for uploading resumes in PDF format and attending assessments. The Admin class also derives from the User class and provides functionalities such as viewing

assessment records, analyzing performance, and downloading reports. These inheritance relationships highlight role-based access control and separation of responsibilities within the system.

The Assessment class plays a central role by managing interview sessions. It stores assessment identifiers, candidate details, and the list of questions generated for evaluation. The QuestionGenerator class is responsible for generating interview questions. It interacts with the AI Service to generate questions dynamically and includes methods to check AI service availability. When the AI service is unavailable, the QuestionGenerator fetches questions from the fallback question bank, ensuring uninterrupted interview assessments. This dual mechanism reflects the system's hybrid AI–fallback design.

The ReportGenerator class handles the creation of PDF assessment reports and the storage of results in Excel format. It receives evaluated assessment data and generates structured reports for documentation and review. The EmailService class is responsible for sending assessment reports to candidates via email, enabling automated communication. The AdminDashboard class provides analytical functionalities, including viewing stored performance records and generating consolidated Excel reports for administrative analysis.

4.5 COLLABORATIVE DIAGRAM

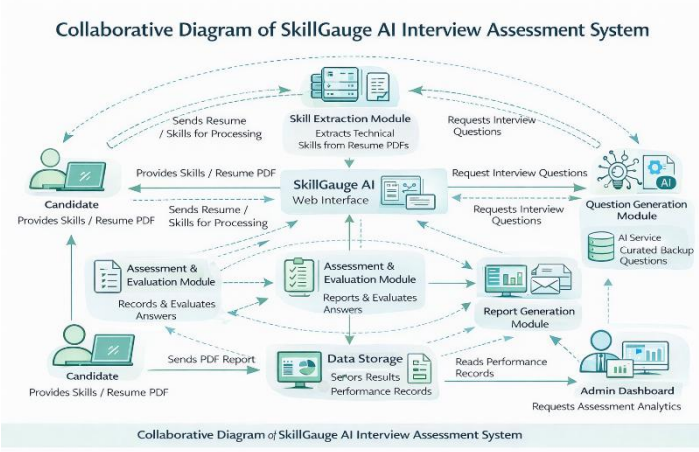


Figure 4.5 Collaborative Diagram

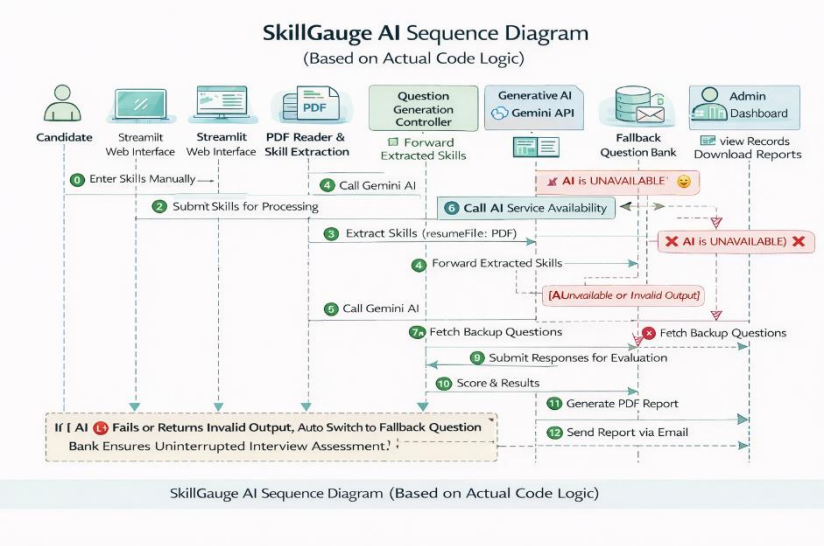
The collaborative diagram of SkillGauge AI – Intelligent Interview Assessment System illustrates how different system components interact with each other to complete the interview assessment workflow. It focuses on message

exchange and collaboration among modules rather than internal structure. The primary participant in the system is the Candidate, who initiates the process by providing technical skills manually or uploading a resume in PDF format through the SkillGauge AI web interface.

Once the candidate submits the input, the Skill Extraction Module collaborates with the web interface to extract relevant technical skills from the resume using text processing techniques. These extracted skills are then forwarded to the Question Generation Module. The system requests interview questions from the Generative AI service when it is available. A key highlight of this collaborative design is the integration of an AI Service Curated Backup Question Bank, which collaborates with the Question Generation Module. When the AI service is unavailable or fails to generate questions, the system automatically retrieves predefined questions from the backup question bank, ensuring uninterrupted interview assessments.

The generated questions are delivered to the Assessment and Evaluation Module, where candidate responses are recorded and evaluated. This module collaborates closely with the Report Generation Module to generate detailed PDF assessment reports. Simultaneously, assessment results and performance records are stored in the Data Storage Module for future reference. The generated PDF report is sent back to the candidate, while the Admin Dashboard collaborates with the data storage and report modules to access assessment records, analyze performance statistics, and request analytics. Overall, the collaborative diagram highlights seamless interaction among system components and emphasizes the hybrid AI–fallback mechanism that guarantees reliability and continuous operation of the SkillGauge AI system.

4.6 SEQUENCE DIAGRAM



SkillGauge AI Sequence Diagram (Based on Actual Code Logic)

Figure 4.6 Sequence Diagram

The sequence diagram illustrates the complete execution flow of the SkillGauge AI – Intelligent Interview Assessment System, showing how different components interact over time to conduct an automated interview assessment. The process begins when the Candidate accesses the Streamlit Web Interface and provides input by either manually entering technical skills or uploading a resume in PDF format.

Once the input is submitted, the Streamlit interface forwards the data for processing. If a resume is uploaded, the PDF Reader and Skill Extraction module is invoked to extract relevant technical skills from the document. These extracted skills are then passed to the Question Generation Controller, which is responsible for generating interview questions.

The Question Generation Controller first interacts with the Generative AI (Gemini API) to generate skill-based multiple-choice questions. The diagram clearly shows a conditional flow where the system checks AI service availability. If the AI service is available and returns valid output, the generated questions are used directly for the assessment. However, if the AI service is unavailable or returns invalid responses, the system automatically switches to the Fallback Question Bank, from

which predefined and curated questions are fetched. This fallback mechanism ensures uninterrupted interview assessment and is a key highlight of the system.

After the questions are prepared, they are presented to the candidate through the Streamlit interface. The candidate submits responses, which are then evaluated by the system to calculate scores and performance results. The Report Generation module generates a detailed PDF assessment report, and the Email Service sends this report to the candidate's registered email address. Finally, assessment data and reports are made available to the Admin Dashboard, where administrators can view records and download reports for performance analysis. The sequence diagram effectively demonstrates the reliability, automation, and hybrid AI–fallback workflow implemented in SkillGauge AI.

CHAPTER 5
SYSTEM IMPLEMENTATION

5.1 INTRODUCTION

The system implementation phase focuses on translating the proposed design of SkillGauge AI – Intelligent Interview Assessment System into a fully functional software application. This phase involves developing individual modules, integrating them seamlessly, and ensuring that the system performs automated interview assessments reliably. The implementation strictly follows the modular architecture discussed in the system design chapter.

SkillGauge AI is implemented as a web-based application using Python and the Streamlit framework. The system integrates Generative AI for dynamic interview question generation while also incorporating a fallback question bank to ensure uninterrupted operation. This hybrid implementation approach guarantees that interview assessments can proceed smoothly even when AI services are unavailable or fail to respond.

The implementation emphasizes automation, reliability, and scalability. Key functionalities such as resume-based skill extraction, AI-driven and fallback-based question generation, automated evaluation, PDF report creation, email delivery, and administrative monitoring are all implemented as independent yet interconnected modules. This chapter explains the environment setup, implementation methodology, module-wise implementation, error handling mechanisms, and the overall system execution flow.

5.2 IMPLEMENTATION ENVIRONMENT

The SkillGauge AI system is implemented using Python as the core programming language due to its rich ecosystem of libraries for web development, artificial intelligence, data processing, and document generation. The application uses the Streamlit framework to build an interactive and responsive web-based user

interface that allows candidates and administrators to interact with the system seamlessly through a browser.

Generative AI integration is achieved using the Gemini API, which is responsible for generating domain-specific interview questions based on extracted skills. To ensure reliability, the system also maintains a locally stored fallback question bank implemented using structured Python data objects. Supporting libraries such as PyPDF2 are used for resume parsing, ReportLab for PDF generation, OpenPyXL for Excel-based data storage, Matplotlib for performance visualization, and smtplib for email automation.

The system is developed and tested on commonly used operating systems such as Windows and Linux. Development tools include VS Code or similar IDEs, Git for version control, and local Python virtual environments. The implementation environment ensures portability, scalability, and ease of deployment across different platforms.

5.3 IMPLEMENTATION METHODOLOGY

The implementation of SkillGauge AI follows a modular and incremental development approach, where each system module is developed, tested, and integrated independently. This approach improves maintainability and allows smooth debugging and enhancement of individual components.

Initially, the user interface and input handling mechanisms were implemented using Streamlit. This was followed by the development of the skill extraction logic, AI-based question generation, and fallback question bank functionality. The fallback mechanism is tightly integrated with the AI module to ensure automatic switching whenever AI services fail or return invalid responses.

Subsequently, automated evaluation, report generation, email delivery, and admin dashboard functionalities were implemented. Each stage was tested using multiple

scenarios, including AI success and AI failure cases, to verify uninterrupted system execution.

5.4 MODULE-WISE IMPLEMENTATION

The system is implemented as a collection of independent yet interconnected modules. Each module performs a specific function within the project evaluation workflow.

1. User Interface Module
2. Input Validation Module
3. AI Evaluation Module
4. Report Generation Module
5. Decision Support Module
6. Output File Generation Module
7. Temporary File Management Module

Each module is described in detail below.

5.4.1 User Interface Module

The User Interface module is implemented using the Streamlit framework to provide an interactive web-based interface for candidates and administrators. Candidates can enter skills manually or upload resumes in PDF format, while administrators can access assessment records through a secure dashboard.

The interface dynamically displays interview questions, progress indicators, and result summaries. Navigation between different stages of assessment is handled using Streamlit session state management.

This module acts as the central interaction layer between users and backend logic, ensuring smooth data flow across all system components.

5.4.2 Input Validation Module

The Input Validation module ensures that users provide valid inputs before starting the assessment. It verifies skill inputs, email formats, and resume file types to prevent invalid data processing.

If mandatory fields are missing or incorrect data is entered, the system displays user-friendly error messages. This prevents system crashes and ensures reliable execution.

Validation is applied consistently for both AI-driven and fallback-based assessment flows.

5.4.3 AI Evaluation Module

The AI Evaluation module is responsible for generating interview questions using the Gemini Generative AI model. Based on the candidate's extracted or manually entered skills, the module generates multiple-choice interview questions dynamically.

Before generating questions, the module checks AI service availability. If the AI responds successfully, generated questions are used for the assessment.

In case of AI failure, timeout, or invalid output, the module automatically activates the fallback question bank, ensuring uninterrupted interview assessment. This hybrid logic is a key highlight of the implementation.

5.4.4 Report Generation Module

The Report Generation module creates detailed assessment reports in PDF format using the ReportLab library. The report includes questions, selected answers, correct answers, scores, and performance summaries.

Reports are generated automatically after assessment completion and follow a structured and professional format suitable for documentation and review.

This module supports transparency and enables candidates to analyze their interview readiness effectively.

5.4.5 Decision Support Module

The Decision Support module analyzes assessment results to determine candidate performance levels. It categorizes performance as excellent, good, or needs improvement based on predefined scoring thresholds.

This module supports both candidates and administrators by providing clear insights into strengths and weaknesses. It also assists recruiters and faculty members in making informed decisions.

The decision logic remains consistent regardless of whether questions are generated using AI or the fallback question bank.

5.4.6 Output File Generation Module

The Output File Generation module handles the creation of downloadable files such as PDF reports and Excel-based assessment records. Excel files store candidate scores, timestamps, and evaluation details.

These files are generated automatically and made available for both candidates and administrators through download options.

This module enhances documentation and long-term record maintenance.

5.4.7 Temporary File Management Module

Temporary files such as uploaded resumes and intermediate report files are managed efficiently during execution. Files are stored only for the duration required for processing.

After use, unnecessary temporary files are discarded to optimize storage usage and improve security.

This module ensures smooth file handling without excessive resource consumption.

5.5 ERROR HANDLING AND EXCEPTION MANAGEMENT

The system includes robust error handling mechanisms to manage unexpected conditions such as invalid file uploads, missing inputs, AI service failures, and email delivery issues. Try–except blocks are used extensively to prevent system crashes.

Meaningful error messages are displayed to guide users in correcting issues. In AI-related failures, the fallback question bank is automatically triggered to maintain uninterrupted operation.

These mechanisms significantly improve system stability and user experience.

5.6 SYSTEM EXECUTION FLOW

The system execution begins with user input through the Streamlit interface. Skills are collected either manually or via resume parsing, followed by question generation using AI or the fallback question bank.

Candidates complete the assessment, responses are evaluated automatically, and performance results are generated. PDF reports are created and emailed to candidates, while assessment data is stored for administrative review.

The execution flow ensures a fully automated, reliable, and scalable interview assessment process supported by a hybrid AI–fallback design.

CHAPTER 6
SYSTEM TESTING

6.1 INTRODUCTION

System testing is a critical phase in the development of **SkillGauge AI – Intelligent Interview Assessment System**, aimed at verifying that the application functions correctly and meets the specified requirements. The testing process ensures that all modules, including AI-driven components and fallback mechanisms, operate reliably under various conditions.

SkillGauge AI involves multiple interconnected components such as user interface, skill extraction, AI-based question generation, fallback question bank, automated evaluation, report generation, and email notification. System testing validates the correctness of these components both individually and collectively.

Special emphasis is given to testing the **AI failure scenarios**, where the system must automatically switch to the predefined question bank. This ensures uninterrupted interview assessments and validates the robustness of the hybrid AI–fallback design.

6.2 OBJECTIVES OF SYSTEM TESTING

The primary objective of system testing is to ensure that SkillGauge AI performs accurate and reliable interview assessments. Testing verifies that all functional requirements, including skill input, resume upload, question generation, evaluation, and reporting, work as intended.

Another objective is to validate the system's ability to handle AI service unavailability gracefully. The fallback question bank is tested to ensure that assessments continue smoothly without user disruption.

Additionally, system testing aims to identify errors, improve performance, ensure usability, and verify data integrity across all system modules.

6.3 TESTING STRATEGY

SkillGauge AI follows a **structured and layered testing strategy**, combining both manual and automated testing approaches. Each module is tested independently before integration, followed by end-to-end system testing.

Testing scenarios include normal execution, invalid inputs, partial failures, and AI service downtime. Special test cases are designed to verify the seamless transition from AI-generated questions to fallback question bank questions.

The testing strategy ensures high reliability, fault tolerance, and consistent performance across different usage scenarios.

6.4 TYPES OF TESTING

6.4.1 Unit Testing

Unit testing is performed to validate individual modules such as skill input handling, resume parsing, question generation, and evaluation logic. Each function is tested with valid and invalid inputs.

The AI question generation function is tested separately from the fallback question bank logic to ensure both paths work correctly.

This testing ensures that each module performs its intended functionality independently.

6.4.2 Integration Testing

Integration testing verifies the interaction between different system modules. The flow from skill extraction to question generation, evaluation, and report generation is tested as a complete process.

Both AI success and AI failure scenarios are tested to ensure smooth integration between the AI module and fallback question bank.

This testing confirms that modules communicate and collaborate correctly.

6.4.3 Functional Testing

Functional testing validates the system against specified functional requirements. Features such as skill entry, resume upload, assessment execution, scoring, report generation, and email delivery are tested.

The fallback question bank is functionally tested by simulating AI service failure and verifying uninterrupted assessment.

This ensures the system behaves as expected from a user perspective.

6.4.4 Input Validation Testing

Input validation testing ensures that invalid or missing inputs are handled properly. This includes testing empty skill inputs, incorrect email formats, unsupported file types, and incomplete assessments.

The system displays appropriate error messages and prevents invalid execution.

This testing improves system reliability and user experience.

6.4.5 Performance Testing

Performance testing evaluates system responsiveness and execution speed. The system is tested with multiple assessments to verify acceptable response times.

Performance is also tested under AI failure conditions to ensure quick fallback to the question bank without delay.

This ensures scalability and smooth operation.

6.4.6 Usability Testing

Usability testing focuses on user interaction and interface clarity. The Streamlit interface is tested for ease of navigation, clarity of instructions, and feedback messages.

Both candidates and administrators are involved in usability evaluation to ensure accessibility and ease of use.

This testing ensures a positive user experience.

6.5 TEST CASE DESCRIPTION

The test case design for the SkillGauge AI system focuses on validating all major functional and non-functional components of the application. Test cases are created to verify correct system behavior for both normal execution and exceptional scenarios. These test cases ensure that the system operates reliably for candidates and administrators under different input conditions.

Initial test cases validate **manual skill entry and resume upload functionality**. The system is tested with valid technical skill inputs and properly formatted PDF resumes to ensure skills are accepted and extracted correctly. Additional test cases handle invalid inputs such as empty skill fields, incorrect email formats, and unsupported file types to confirm that the system displays appropriate validation messages and prevents incorrect execution.

Question generation test cases are designed to evaluate both **AI-based and fallback-based workflows**. When the Generative AI service is available, the system is tested to confirm that AI-generated interview questions are displayed correctly. In scenarios where the AI service is unavailable or fails to respond, test cases verify that the system automatically switches to the predefined fallback question bank without interrupting the assessment process.

Further test cases focus on **assessment execution and evaluation logic**. These include validating answer submission, automatic score calculation, and performance classification. The system is tested to ensure accurate evaluation of responses and correct computation of scores regardless of whether questions originate from the AI model or the fallback question bank.

Report generation and communication functionalities are also tested extensively. Test cases validate that PDF assessment reports are generated correctly after assessment completion and that reports are successfully sent to the candidate's registered email address. Additionally, test cases verify that assessment data is stored correctly and accessible through the admin dashboard for performance analysis and record management.

Overall, the test cases confirm that SkillGauge AI meets functional requirements, handles AI failures gracefully through fallback mechanisms, and delivers consistent, reliable interview assessment results.

6.6 TEST RESULTS AND ANALYSIS

The testing process confirmed that SkillGauge AI performs all intended functions accurately. Manual and resume-based skill inputs were processed correctly, and interview assessments were conducted without errors.

AI-based question generation worked reliably when the AI service was available. In all simulated AI failure scenarios, the fallback question bank was activated automatically, ensuring uninterrupted assessments. This validated the robustness of the hybrid design.

Performance testing showed acceptable response times, and usability testing indicated that the interface was user-friendly. Overall, the system met all functional and non-functional requirements successfully.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 CONCLUSION

The **SkillGauge AI – Intelligent Interview Assessment System** successfully addresses the challenges associated with traditional interview and skill evaluation processes in both recruitment and academic environments. By automating interview question generation, assessment, evaluation, and reporting, the system reduces manual effort, improves consistency, and ensures unbiased assessment of candidates and students. The web-based implementation using Streamlit provides an accessible and user-friendly platform for conducting technical assessments.

A key strength of SkillGauge AI is its **hybrid design**, which integrates Generative AI with a **fallback question bank mechanism**. This ensures uninterrupted interview assessments even when AI services are unavailable or fail to respond. The system's ability to dynamically switch between AI-generated questions and curated fallback questions enhances reliability and robustness, making it suitable for real-world deployment where external AI dependencies may not always be stable.

The system also provides automated performance analysis, PDF report generation, and email-based result delivery, improving transparency and documentation. The inclusion of an administrative dashboard enables effective monitoring and analysis of assessment records. Overall, SkillGauge AI demonstrates how artificial intelligence can be effectively combined with traditional rule-based mechanisms to create a scalable, reliable, and efficient interview assessment platform for students, job seekers, educational institutions, and recruiters.

7.2 FUTURE WORK

The SkillGauge AI system provides a strong foundation for intelligent interview assessment; however, several enhancements can be implemented in future to further improve its capabilities. One major area of improvement is the integration of **advanced adaptive questioning**, where the difficulty level of questions dynamically adjusts based on the candidate's real-time performance. This would allow more personalized and in-depth assessment of technical proficiency.

Future versions of the system can incorporate **coding-based assessments and practical problem-solving tasks** in addition to multiple-choice questions. This enhancement would enable more comprehensive evaluation of hands-on programming skills, making the platform more suitable for technical recruitment and skill certification. Support for additional file formats and multimedia-based assessments can also be introduced.

Scalability and intelligence can be further enhanced by deploying the system on **cloud infrastructure** with role-based access control and database-backed storage. Integration of **machine learning analytics** for long-term performance tracking, recommendation systems for skill improvement, and multilingual support would broaden the system's applicability. These enhancements would transform SkillGauge AI into a complete, enterprise-level intelligent assessment platform.

APPENDIX-I

SAMPLE CODE

app.py

```
import streamlit as st
import os, json, re, smtplib, random
from datetime import datetime
from email.message import EmailMessage
import pandas as pd
import matplotlib.pyplot as plt
from PyPDF2 import PdfReader
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.enums import TA_CENTER
from reportlab.lib.units import inch
from openpyxl import Workbook, load_workbook
from google import genai
# ====================================================
# PAGE CONFIG & STYLING
# ====================================================
st.set_page_config(
    page_title="SkillGauge AI – Interview Assessment",
    layout="wide",
    initial_sidebar_state="collapsed"
)
```

```
# ================================================

# CUSTOM CSS (ONLY TEXT COLOR FIXED)

# ================================================

st.markdown("""

<style>

* {

    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;

}

html, body, [data-testid="stAppViewContainer"] {

    background: linear-gradient(135deg, #667eea 0%, #764ba2 50%, #f093fb 100%) !important;

    min-height: 100vh;

}

/* Headings remain white */

h1, h2, h3 {

    color: #ffffff;

    font-weight: 700;

}


/* ------------------------------

  BLACK TEXT VISIBILITY FIX

------------------------------ */

/* Input text */

.stTextInput input,

.stTextArea textarea,
```

```
.stSelectbox select {
    color: #000000 !important;
    background-color: rgba(255,255,255,0.85) !important;
}
/* Placeholder text */
.stTextInput input::placeholder,
.stTextArea textarea::placeholder {
    color: #000000 !important;
    opacity: 0.7;
}
/* Labels */
label,
.stRadio label,
.stSelectbox label {
    color: #000000 !important;
    font-weight: 600;
}
RESULT_DIR = "results"
PDF_DIR = "pdf_reports"
EXCEL_PATH = os.path.join(RESULT_DIR, "student_scores.xlsx")
os.makedirs(RESULT_DIR, exist_ok=True)
os.makedirs(PDF_DIR, exist_ok=True)
# ================================================
# GEMINI CONFIG
```

```
# ====================================================

# client = genai.Client(api_key="AIzaSyCS9x37uOK1CJkgrBnDdeU93OEjUDtLtwE")

MODEL_NAME = "models/gemini-flash-latest"

# ====================================================

# EMAIL CONFIG

# ====================================================

SENDER_EMAIL = "noreply.civicconnect@gmail.com"

APP_PASSWORD = "hvpwfvoouycoaqgo"

# ====================================================

# FALLBACK QUESTION BANK (50 Questions - 25 Random Selected if AI Fails)

# ====================================================

QUESTION_BANK = [

    # Python (7 questions)

    {

        "q": "What is the output of: print(type([]))",

        "options": {"A": "<class 'list'>", "B": "<class 'tuple'>", "C": "<class 'dict'>", "D": "<class
'array'>"},

        "answer": "A",

        "domain": "Python"

    },

    {

        "q": "Which keyword is used to create a function in Python?",

        "options": {"A": "function", "B": "def", "C": "define", "D": "func"},

        "answer": "B",

        "domain": "Python"
```

```
    },
{
        "q": "What does len() function return for a string 'hello'?",
        "options": {"A": "4", "B": "5", "C": "6", "D": "None"},
        "answer": "B",
        "domain": "Python"
    },
    {
        "q": "How do you access the first element of a list in Python?",
        "options": {"A": "list[1]", "B": "list[0]", "C": "list.first()", "D": "list[:]"},
        "answer": "B",
        "domain": "Python"
    },
    {
        "q": "What is the correct syntax for creating a dictionary in Python?",
        "options": {"A": "{key: value}", "B": "[key: value]", "C": "(key: value)", "D": "key: value"},
        "answer": "A",
        "domain": "Python"
    },
    {
        "q": "Which method is used to add an element to a list?",
        "options": {"A": "add()", "B": "insert()", "C": "append()", "D": "push()"},
        "answer": "C",
        "domain": "Python"
```

```
    }
]
# ==========================================================
# SESSION STATE
# ==========================================================
if "step" not in st.session_state:
    st.session_state.update({
        "step": 0,
        "mcqs": [],
        "answers": {},
        "score": 0,
        "email": "",
        "evaluated": False,
        "admin_logged": False,
        "ai_used": False  # Track if AI or fallback was used
    })
# ==========================================================
# SAFE JSON EXTRACTION (FLASH-SAFE)
# ==========================================================
def extract_json_flexible(text):
    text = re.sub(r"```json|```", "", text).strip(
    # Try direct JSON
    try:
        data = json.loads(text)
```

```
        if isinstance(data, list):
            return data
        if isinstance(data, dict):
            for k in ("mcqs", "questions"):
                if k in data and isinstance(data[k], list):
                    return data[k]
    except:
        pass
    # Extract JSON array from text
    match = re.search(r"\[[\s\S]*\]", text)
    if match:
        try:
            return json.loads(match.group())
        except:
            pass
    return None
ef extract_mcqs(parsed):
    if isinstance(parsed, dict):
        for k in ("mcq", "mcqs", "questions"):
            if k in parsed and isinstance(parsed[k], list):
                return parsed[k]
    if isinstance(parsed, list):
        return parsed
    return None
```

```
# ==========================================================
# SKILL EXTRACTION
# ==========================================================
def extract_skills_from_resume(file):
    reader = PdfReader(file)
    text = " ".join(p.extract_text() or "" for p in reader.pages)
    prompt = f"""
Extract ONLY technical skills, programming languages, frameworks,
tools and domains (comma separated).
Resume:
{text[:6000]}
"""
    try:
        res = client.models.generate_content(model=MODEL_NAME, contents=prompt)
        return res.text.strip()
    except:
        st.warning("⚠️ AI unavailable. Please enter skills manually.")
        return ""
def generate_interview_mcqs(skills):
    """
    AI-first MCQ generation with guaranteed fallback
    """
    prompt = f"""
Return ONLY a valid JSON ARRAY.
```

```
FORMAT:
[
  {{
    "q": "Question",
    "options": {{
      "A": "...",
      "B": "...",
      "C": "...",
      "D": "..."
    }},
    "answer": "A"
  }}
]
Generate exactly 25 interview MCQs based on:
{skills}
"""
for attempt in range(2):
        try:
            res = client.models.generate_content(
                model=MODEL_NAME,
                contents=prompt
            )
            mcqs = extract_json_flexible(res.text)
            if mcqs and isinstance(mcqs, list) and len(mcqs) >= 25:
```

```
            st.session_state.ai_used = True
            return mcqs[:25]
        except Exception as e:
            print(f"[AI ERROR] Attempt {attempt+1}:", e)
    # FALLBACK (UNCHANGED)
    st.warning("⚠️ AI service temporarily unavailable. Using curated question bank.")
    st.session_state.ai_used = False
    return random.sample(QUESTION_BANK, 25)
# ============================================================
# SEND EMAIL WITH PDF REPORT
# ============================================================
def send_email_report(recipient_email, pdf_path, student_name="Student"):
    """Send assessment report via email"""
    try:
        msg = EmailMessage()
        msg['Subject'] = "Your SkillGauge AI Assessment Report"
        msg['From'] = SENDER_EMAIL
        msg['To'] = recipient_email

        body = f"""
Hi {student_name},
Thank you for completing your SkillGauge AI Assessment!
Your personalized interview assessment report is attached. This report includes:
- Questions asked
```

- Your answers

- Correct answers

- Overall score and performance analysis

We hope this helps you prepare for your interview. If you have any questions, please feel free to reach out.

Best regards,

SkillGauge AI Team

"""

```
        msg.set_content(body)
        # Attach PDF
        with open(pdf_path, 'rb') as attachment:
            msg.add_attachment(
                attachment.read(),
                maintype='application',
                subtype='pdf',
                filename=os.path.basename(pdf_path)
            )
        # Send via Gmail
        with smtplib.SMTP_SSL('smtp.gmail.com', 465) as smtp:
            smtp.login(SENDER_EMAIL, APP_PASSWORD)
            smtp.send_message(msg)
        return True
    except Exception as e:
        print(f"[v0] Email send failed: {str(e)}")
        return False
```

```
# ==================================================
# SAVE RESULT
# ==================================================
def save_to_excel(email, score):
    if not os.path.exists(EXCEL_PATH):
        wb = Workbook()
        ws = wb.active
        ws.append(["Email", "Score", "Max Marks", "Date", "Time"])
        wb.save(EXCEL_PATH)

    wb = load_workbook(EXCEL_PATH)
    ws = wb.active
    now = datetime.now()
    ws.append([email, score, 25, now.strftime("%Y-%m-%d"), now.strftime("%H:%M:%S")])
    wb.save(EXCEL_PATH)
# ==================================================
# PDF GENERATION
# ==================================================
def generate_pdf(email, mcqs, answers, score):
    path = os.path.join(PDF_DIR, f"result_{int(datetime.now().timestamp())}.pdf")
    styles = getSampleStyleSheet()
    styles.add(ParagraphStyle(
        name="TitleX",
        alignment=TA_CENTER,
```

```
        fontSize=20,
        textColor="#1e3a5f"
    ))
    doc = SimpleDocTemplate(path)
    story = []
    story.append(Paragraph("SkillGauge AI – Interview Assessment Report", styles["TitleX"]))
    story.append(Spacer(1, 12))
    story.append(Paragraph(f"<b>Email:</b> {email}", styles["Normal"]))
    story.append(Paragraph(f"<b>Report Date:</b> {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}", styles["Normal"]))
    story.append(Spacer(1, 12))
    for i, q in enumerate(mcqs, 1):
        story.append(Spacer(1, 10))
        story.append(Paragraph(f"<b>{i}. {q['q']}</b>", styles["Normal"]))
        user_ans = answers.get(i, "Not answered")
        correct_ans = q["answer"]
        is_correct = user_ans == correct_ans
        for k, v in q["options"].items():
            marker = "✓" if (k == correct_ans and is_correct) else "✗" if (k == user_ans and not is_correct) else ""
            story.append(Paragraph(f"{k}) {v} {marker}", styles["Normal"]))
        story.append(Spacer(1, 6))
    story.append(Spacer(1, 12))
    story.append(Paragraph(f"<b>FINAL SCORE: {score}/25</b>", styles["Normal"]))
    percentage = (score / 25) * 100
```

```
    story.append(Paragraph(f"<b>Percentage: {percentage:.1f}%</b>", styles["Normal"]))
    doc.build(story)
    return path
    # ==========================================================
# ADMIN LOGIN
# ==========================================================
if st.session_state.step == "admin_login":
    st.markdown("## 🔐 Admin Login")
    col1, col2, col3 = st.columns([1, 1, 1])
    with col2:
        with st.container():
            st.markdown("---")
            username = st.text_input("Username", placeholder="Enter username")
            password = st.text_input("Password", type="password", placeholder="Enter password")
            col_login, col_back = st.columns(2)
            with col_login:
                if st.button("✓ Login", use_container_width=True):
                    if username == "admin" and password == "admin123":
                        st.session_state.admin_logged = True
                        st.session_state.step = "admin_dashboard"
                        st.rerun()
                    else:
                        st.error("❌ Invalid credentials. Please try again.")
            with col_back:
```

```
            if st.button("← Back", use_container_width=True):
                st.session_state.step = 0
                st.rerun()
# ========================================================
# ADMIN DASHBOARD
# ========================================================
elif st.session_state.step == "admin_dashboard":
    st.markdown("## 📊 Admin Dashboard")
    st.markdown("---")
    if not os.path.exists(EXCEL_PATH):
        st.warning("⚠️ No assessment data yet.")
    else:
        df = pd.read_excel(EXCEL_PATH)
        # Summary Metrics
        col1, col2, col3, col4 = st.columns(4)
        with col1:
            st.metric("Total Assessments", len(df))
        with col2:
            st.metric("Avg Score", f"{df['Score'].mean():.1f}/25")
        with col3:
            st.metric("Highest Score", f"{df['Score'].max()}/25")
        with col4:
            st.metric("Lowest Score", f"{df['Score'].min()}/25")
        st.markdown("---")
```

```
# Data Table
st.subheader(" 📋 Assessment Records")
st.dataframe(df, use_container_width=True)
st.markdown("---")
# Download Button
with open(EXCEL_PATH, "rb") as f:
    st.download_button(
        label=" 📥 Download Excel Report",
        data=f.read(),
        file_name="student_scores.xlsx",
        mime="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
        use_container_width=True
    )
st.markdown("---")
# Individual Analysis
st.subheader(" 📈 Student Performance Analysis")
email = st.selectbox("Select Student Email", df["Email"].unique())
if email:
    latest = df[df["Email"] == email].iloc[-1]
    col1, col2, col3 = st.columns(3)
    with col1:
        st.metric("Score", f"{int(latest['Score'])}/25")
    with col2:
        percentage = (latest['Score'] / 25) * 100
```

```
        st.metric("Percentage", f"{percentage:.1f}%")
    with col3:
        st.metric("Assessment Date", latest['Date'])
    # Pie Chart - Smaller Size
    fig, ax = plt.subplots(figsize=(4, 3))
    fig.patch.set_alpha(0)
    ax.patch.set_alpha(0)
    colors = ['#10b981', '#ef4444']
    wedges, texts, autotexts = ax.pie(
        [latest["Score"], 25 - latest["Score"]],
        labels=["✓ Correct", "✗ Wrong"],
        autopct="%1.1f%%",
        startangle=90,
        colors=colors,
        explode=(0.05, 0.05),
        shadow=False,
        textprops={'fontsize': 10, 'weight': 'bold', 'color': 'white'}
    )
    for autotext in autotexts:
        autotext.set_color('white')
        autotext.set_fontsize(10)
    ax.axis("equal")
    st.pyplot(fig, use_container_width=False)
st.markdown("---")
```

```
        # Logout Button
        if st.button(" 🚪 Logout", use_container_width=True):
            st.session_state.step = 0
            st.session_state.admin_logged = False
            st.rerun()
# ============================================================
# STUDENT FLOW - STEP 0: Input
# ============================================================
elif st.session_state.step == 0:
    st.markdown("## Welcome to SkillGauge AI")
    st.markdown("Professional Interview Assessment Platform")
    st.markdown("---")
    with st.container():
        method = st.radio(
            "How would you like to provide your profile?",
            [" 📝 Enter Skills Manually", " 📄 Upload Resume (PDF)"],
            horizontal=True
        )
        st.markdown("---")
        if method == " 📝 Enter Skills Manually":
            st.subheader("Enter Your Technical Skills")
            skills = st.text_area(
                "List your skills (comma separated)",
                placeholder="e.g. Python, JavaScript, React, SQL, AWS, Machine Learning",
```

```
            height=100
        )
        st.markdown("**Example:** Python, JavaScript, React, SQL, AWS, Cloud Computing")
        st.markdown("---")
        st.subheader("Contact Information")
        email = st.text_input(
            "Your Email Address",
            placeholder="your.email@example.com"
        )
        st.session_state.email = email
    else:  # Resume Upload
        st.subheader("Upload Your Resume")
        file = st.file_uploader("Choose a PDF file", type="pdf")
        if file:
            with st.spinner("📖 Extracting skills from resume..."):
                skills = extract_skills_from_resume(file)
            skills = st.text_area(
                "Edit extracted skills if needed",
                value=skills,
                height=100
            )
            st.markdown("---")
            st.subheader("Confirm Contact Information")
            email = st.text_input(
```

```
            "Your Email Address",
            placeholder="your.email@example.com"
        )
        st.session_state.email = emai
    st.markdown("---")
    # Validation and Submit
    col1, col2 = st.columns(2)
    with col1:
        if st.button("🚀 Start Interview", use_container_width=True):
            if not skills.strip():
                st.error("❌ Please enter your skills.")
            elif not email.strip():
                st.error("❌ Please enter your email address.")
            else:
                with st.spinner("🤖 Generating interview questions..."):
                    mcqs = generate_interview_mcqs(skills)

                if not mcqs:
                    st.error("❌ Failed to generate questions. Please retry.")
                else:
                    st.session_state.mcqs = mcqs
                    ai_status = "🤖 AI-Generated" if st.session_state.ai_used else "📚 Curated
Questions"
                    st.success(f"✅ Questions ready! ({ai_status})")
```

```
                st.session_state.step = 1
                st.rerun()
        with col2:
            st.info("💡 Tip: More specific skills lead to better interview questions!")
# ============================================================
# STUDENT FLOW - STEP 1: MCQ Assessment
# ============================================================
elif st.session_state.step == 1:
    st.markdown("## 📝 Interview Assessment")
    st.markdown(f"**Total Questions:** {len(st.session_state.mcqs)}")
    ai_status = "🤖 AI-Generated" if st.session_state.ai_used else "📚 Curated Questions"
    st.markdown(f"**Question Source:** {ai_status}")
    st.markdown("---")
    progress = 0
    for i, q in enumerate(st.session_state.mcqs, 1):
        with st.container():
            # Progress indicator
            progress_percent = (i / len(st.session_state.mcqs)) * 100
            st.progress(progress_percent / 100)
            # Question number and text
            st.markdown(f"### Question {i} of {len(st.session_state.mcqs)}")
            st.markdown(f"**{q['q']}**", unsafe_allow_html=True)
            # Options
            options_list = list(q["options"].keys())
```

```
            default_index = 0
            if i in st.session_state.answers:
                try:
                    default_index = options_list.index(st.session_state.answers[i])
                except:
                    default_index = 0
            selected = st.radio(
                "Select your answer:",
                options_list,
                format_func=lambda x: f"{x}) {q['options'][x]}",
                index=default_index,
                key=f"q{i}",
                horizontal=False
            )
            st.session_state.answers[i] = selected
            st.markdown("---")
    # Submit Button
    if st.button("✓ Submit Assessment", use_container_width=True):
        st.session_state.step = 2
        st.rerun()
        with col_submit:
            submitted = st.form_submit_button("🚀 Evaluate Project", use_container_width=True
        # ==========================================================
# STUDENT FLOW - STEP 2: Results
```

```
# ========================================================
elif st.session_state.step == 2:
    score = sum(
        1 for i, q in enumerate(st.session_state.mcqs, 1)
        if st.session_state.answers.get(i) == q["answer"]
    )
    percentage = (score / 25) * 100

    pdf = generate_pdf(st.session_state.email, st.session_state.mcqs, st.session_state.answers,
score)
    save_to_excel(st.session_state.email, score)
    # Try to send email
    email_sent = send_email_report(st.session_state.email, pdf)
    # Result Header
    st.markdown("## ✅ Assessment Complete!")
    st.markdown("---"
    if email_sent:
        st.success("📧 Report sent to your email successfully!")
    else:
        st.info("📧 PDF report generated (email sending unavailable)")
    st.markdown("---")
    # Score Display
    col1, col2, col3 = st.columns(3)
    with col1:
```

```
    st.metric("Your Score", f"{score}/25")
with col2:
    st.metric("Percentage", f"{percentage:.1f}%")
with col3:
    if percentage >= 80:
        status = "🟢 Excellent"
    elif percentage >= 60:
        status = "🟡 Good"
    else:
        status = "🔴 Needs Improvement"
    st.metric("Performance", status)
st.markdown("---")
# Performance Analysis
col1, col2 = st.columns(2)
with col1:
    st.subheader("Performance Chart")
    fig, ax = plt.subplots(figsize=(4, 3.5))
    fig.patch.set_alpha(0)
    ax.patch.set_alpha(0)
    colors = ['#10b981', '#ef4444']
    wedges, texts, autotexts = ax.pie(
        [score, 25 - score],
        labels=["✓ Correct", "✗ Wrong"],
        autopct="%1.1f%%",
```

```
        startangle=90,
        colors=colors,
        explode=(0.05, 0.05),
        shadow=False,
        textprops={'fontsize': 10, 'weight': 'bold', 'color': 'white'}
    )
    for autotext in autotexts:
        autotext.set_color('white')
        autotext.set_fontsize(10)
    for text in texts:
        text.set_color('white')
        text.set_fontsize(10)
    ax.axis("equal")
    st.pyplot(fig, use_container_width=False)
with col2:
    st.subheader("Answer Review")
    correct = 0
    for i, q in enumerate(st.session_state.mcqs, 1):
        user_ans = st.session_state.answers.get(i)
        correct_ans = q["answer"]
        is_correct = user_ans == correct_ans
        if is_correct:
            st.success(f"Q{i}: ✓ Correct")
        else:
```

```
            st.error(f"Q{i}: ✗ Wrong (Your: {user_ans}, Correct: {correct_ans})")
st.markdown("---")
# Download Report
with open(pdf, "rb") as f:
    st.download_button(
        label="📄 Download Full Report (PDF)",
        data=f.read(),
        file_name=os.path.basename(pdf),
        mime="application/pdf",
        use_container_width=True
    )
st.markdown("---")
# Reset Button
if st.button("🔄 Take Another Assessment", use_container_width=True):
    st.session_state.step = 0
    st.session_state.mcqs = []
    st.session_state.answers = {}
    st.session_state.score = 0
    st.session_state.ai_used = False
    st.rerun()
```

APPENDIX II

SAMPLE OUTPUT

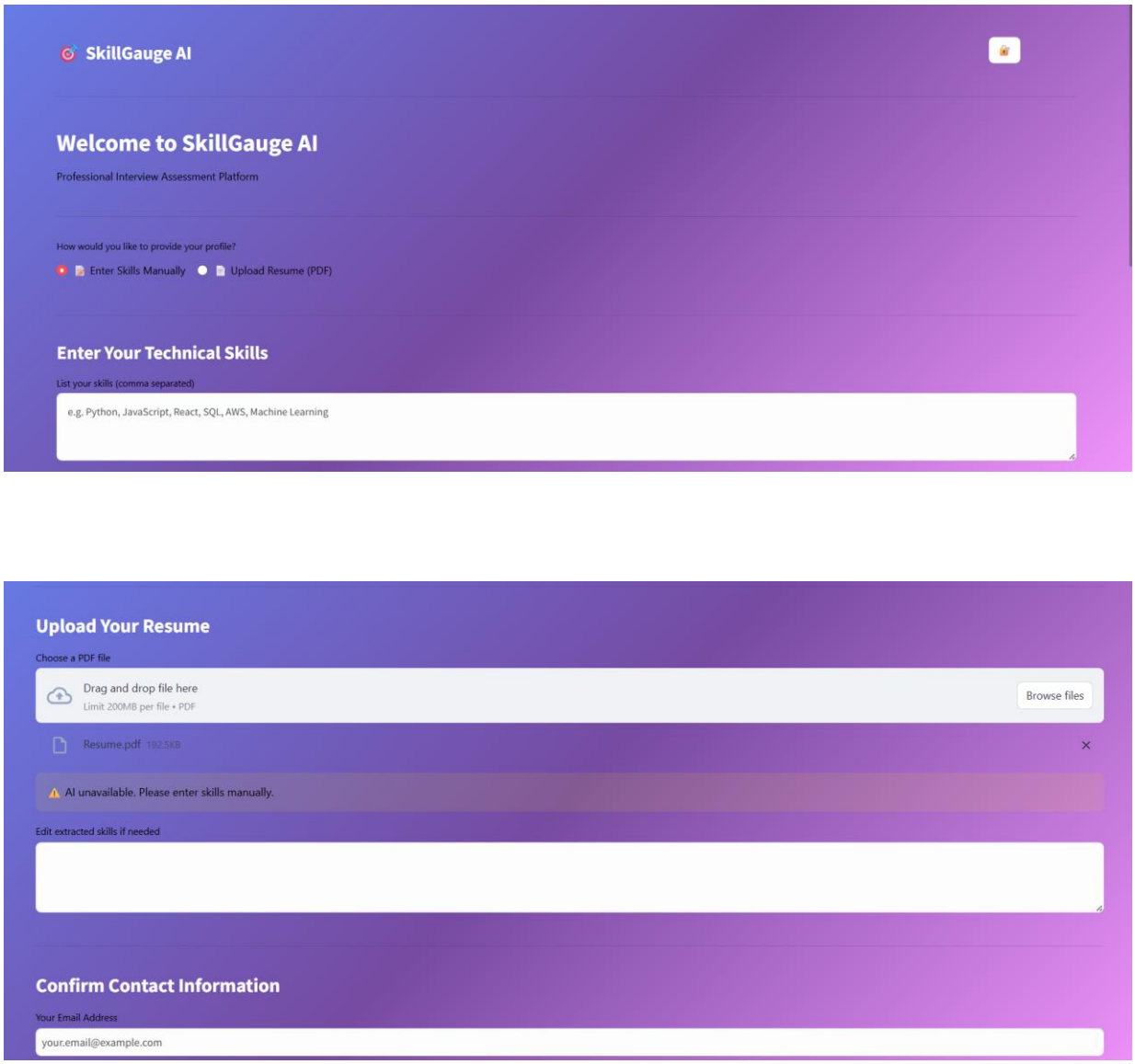


Figure A II.1 & 2 Home Page for Uploading Skills

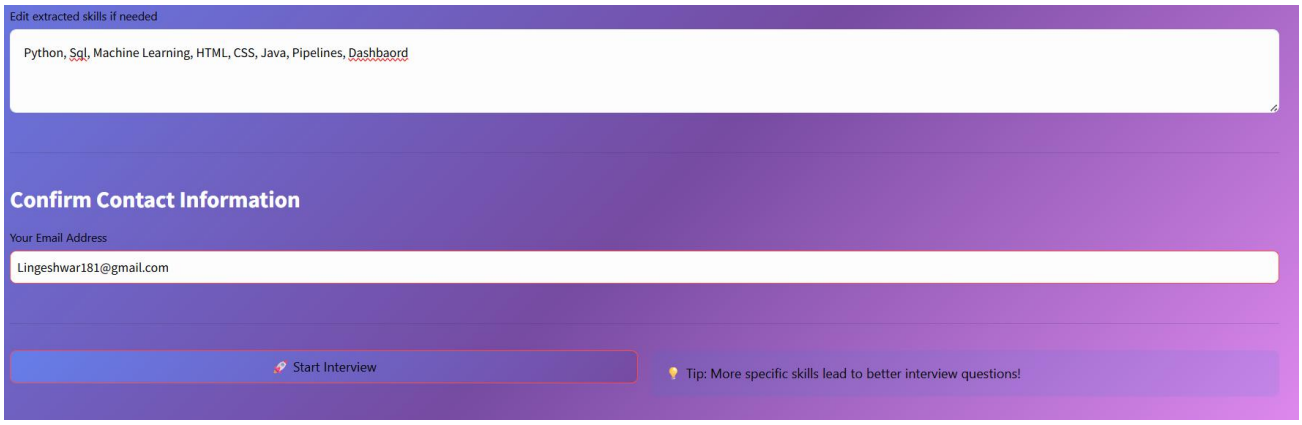


Figure A II.3 Skills Extraction

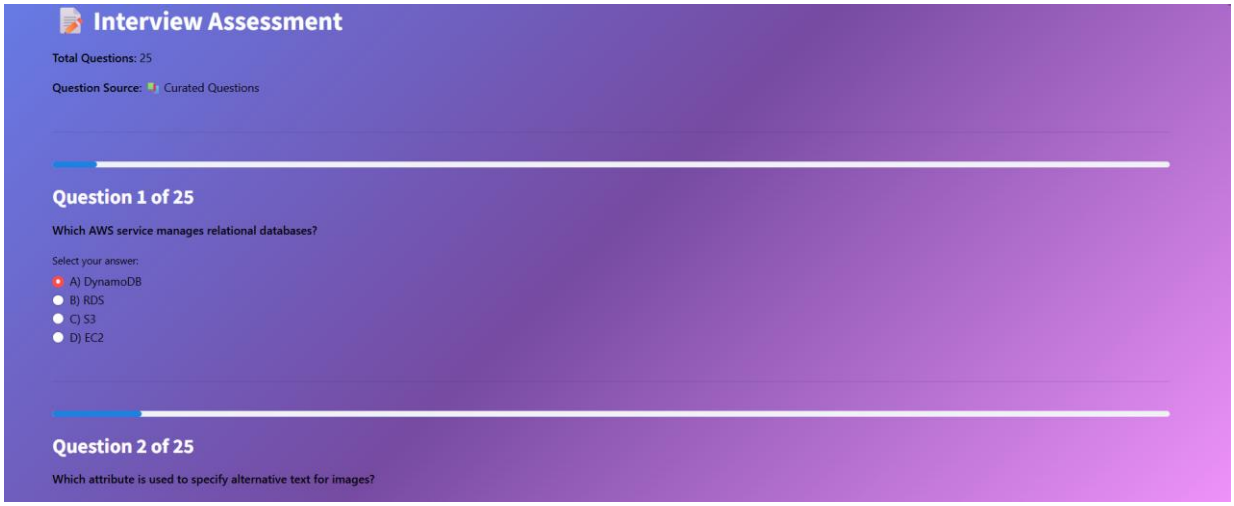


Figure A II.4 Interview Questions Generation

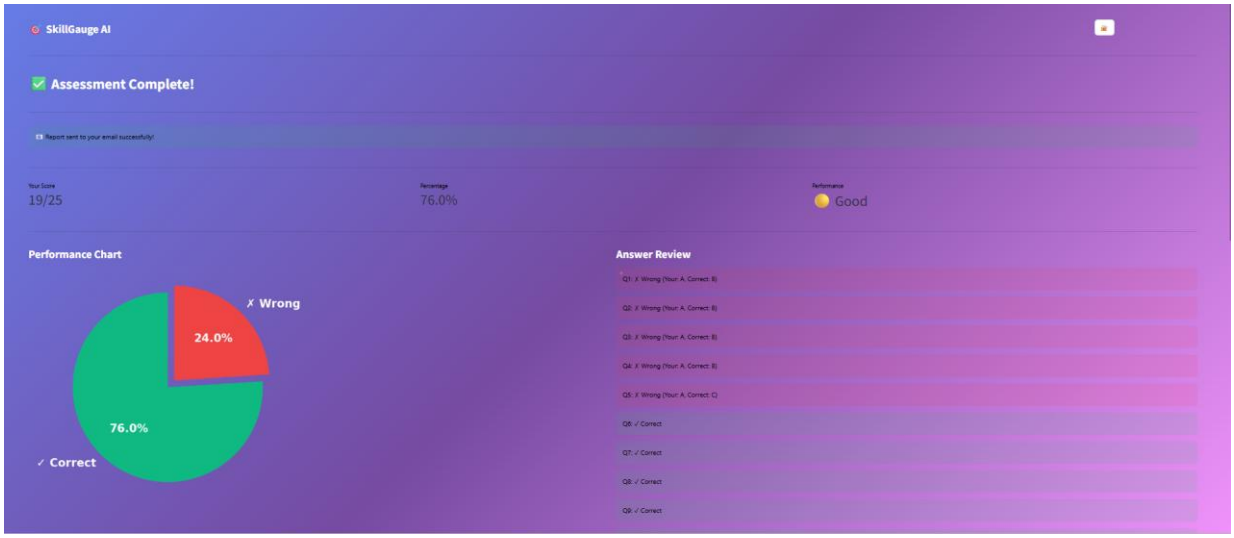


Figure A II.5 Evaluation Output

18. How do you add a comment in JavaScript?
A) # comment
B) // comment or /* */ ✓
C) -- comment
D)

19. Which SQL keyword is used to filter results?
A) FILTER
B) WHERE ✓
C) SEARCH
D) FIND

20. Which method is used to create a string from an object in Java?
A) valueOf()
B) toString() ✓
C) stringify()
D) convert()

21. What does 'this' keyword refer to in Java?
A) Current object ✓
B) Parent class
C) Static variable
D) Method parameter

22. What does CSS stand for?
A) Computer Style Sheet
B) Cascading Style Sheet ✓
C) Creative Style Sheet
D) Coded Style Sheet

23. How do you access the first element of a list in Python?
A) list[1]
B) list[0] ✓
C) list.first()
D) list[:]

24. What is the correct syntax for creating a dictionary in Python?
A) {key: value} ✓
B) [key: value]
C) (key: value)
D) key: value

25. What does feature scaling do?
A) Removes duplicate features
B) Normalizes feature values to similar range ✓
C) Increases model accuracy
D) Reduces computation time

FINAL SCORE: 19/25
Percentage: 76.0%

Figure A II.6 Detailed PDF Report

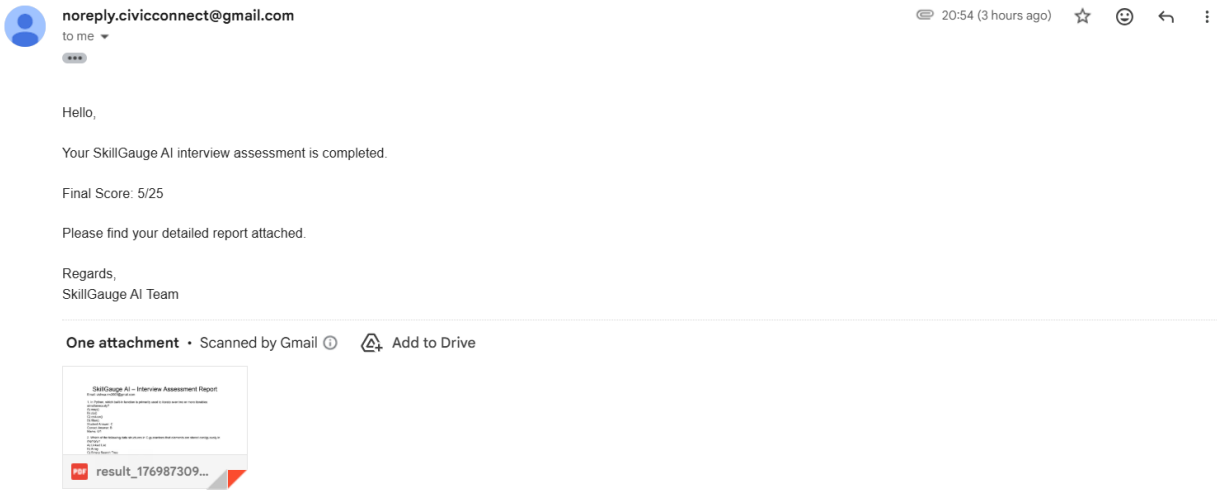


Figure A II.7 Mail with PDF

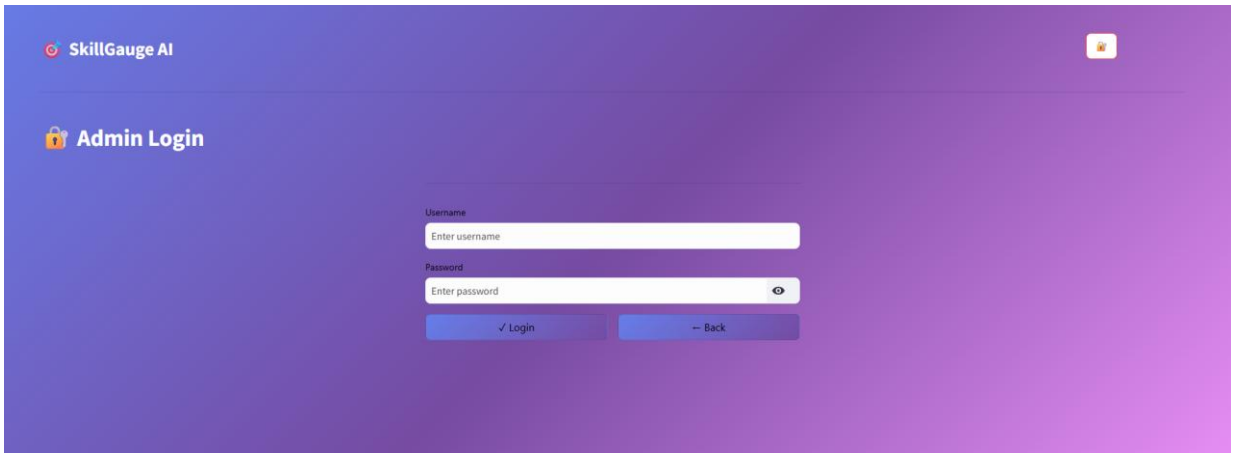
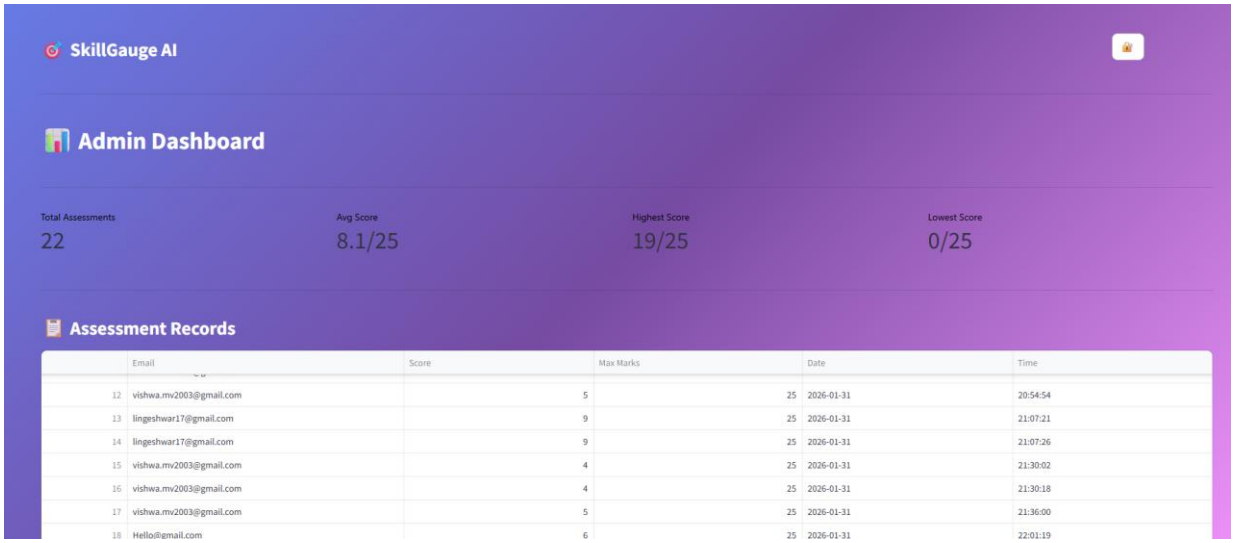


Figure A II.8 Admin Login



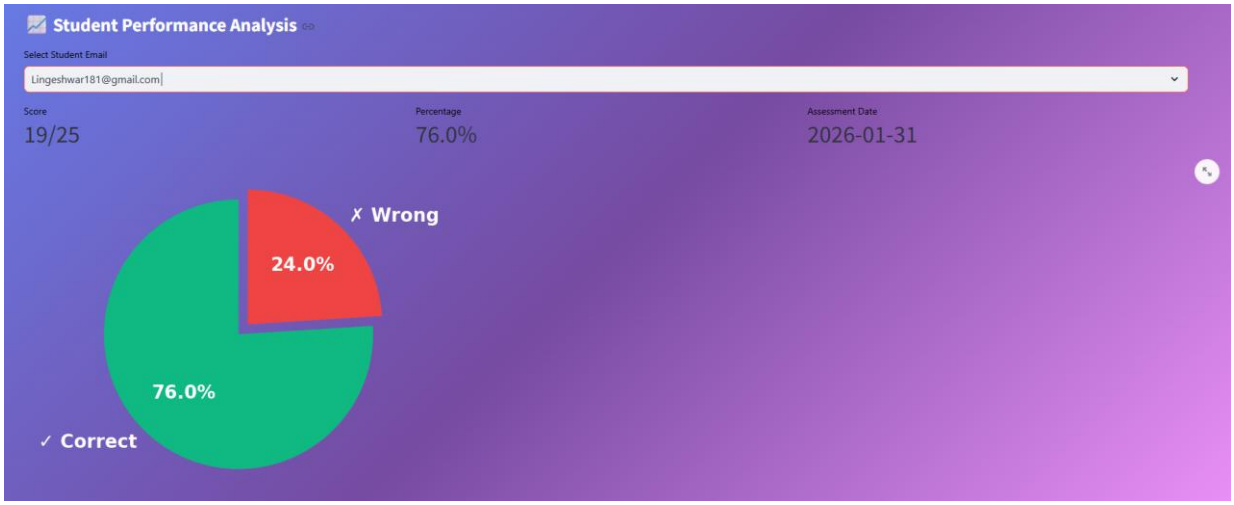


Figure A II.9 & 10 Admin Dashboard

REFERENCES

[1] S. Maity, A. Deroy (2024), “The Future of Learning in the Age of Generative AI: Automated Question Generation and Assessment with Large Language Models”, arXiv Preprint, PP. 1–15.

[2] U. Mittal, S. Sai, V. Chamola, D. Sangwan (2024), “A Comprehensive Review on Generative AI for Education”, IEEE Access, Vol. 12, PP. 34567–34589.

[3] J. Álvarez Ariza, M. Benitez Restrepo, C. Hernández Hernández (2025), “Generative AI in Engineering and Computing Education: A Scoping Review of Empirical Studies and Educational Practices”, IEEE Access, Vol. 13, PP. 11234–11258.

[4] S. Sharma, P. Mittal, M. Kumar, V. Bhardwaj (2025), “The Role of Large Language Models in Personalized Learning”, Education and Information Technologies, Springer, PP. 1–20.

[5] E. Frankford, C. Sauerwein, P. Bassner, S. Krusche, R. Breu (2024), “AI Tutoring in Software Engineering Education: Experiences with Large Language Models”, Proceedings of the IEEE/ACM International Conference on Software Engineering, PP. 1–10.

[6] I. Naim, M. I. Tanveer, D. Gildea, M. E. Hoque (2018), “Automated Analysis and Prediction of Job Interview Performance”, IEEE Transactions on Affective Computing, Vol. 9, No. 2, PP. 191–204.

[7] S. S. Y. Tun, H. Li, M. Wu (2023), “Multimodal Transfer Learning for Oral Presentation Assessment”, IEEE Access, Vol. 11, PP. 88921–88934.

[8] J. Barambones, A. Fernández, L. Otaegui (2024), “ChatGPT for Learning HCI Techniques: A Case Study on Interviews”, IEEE Transactions on Learning Technologies, Vol. 17, No. 1, PP. 45–58.

[9] B. Putra, R. Kurniawan, M. H. Habaebi (2024), “Fair Automated Video Interview Assessment Using Deep Learning Models”, IEEE Access, Vol. 12, PP. 56789–56804.

[10] P. Chavan, S. Jadhav (2026), “AI-Based Mock Interview and Evaluation System”, International Journal of Scientific Research in Science, Engineering and Technology (IJSRSET), Vol. 12, No. 1, PP. 101–110.